P3104R0
Bit permutations

Published Proposal, 2024-01-26

This version:
https://eisenwave.github.io/cpp-proposals/bit-permutations.html

Author:
Jan Schultke

Audience:
SG18, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

This proposal adds bit permutation functions to the bit manipulation library.

Table of Contents

§ 1. Introduction

The C++ bit manipulation library in `<bit>` is an invaluable abstraction from hardware operations. Functions like `countl_zero` help the programmer avoid use of intrinsic functions or inline assembly.

However, there are still a few operations which are non-trivial to implement in software and have widely available hardware support. Therefore, I propose to expand the bit manipulation library with multiple bit permutation functions described below. Even without hardware support, these functions provide great utility and/or help the developer write more expressive code.

§ 2. Proposed features

I propose to add the following functions to the bit manipulation library (`<bit>`).

> NOTE: All constraints are exposition-only. The function bodies contain a naive implementation that merely illustrates the behavior.

> NOTE: See § 3.3 Hardware support for the corresponding hardware instructions.

§ 2.1. `bit_reverse`

```
template<unsigned_integral T>
constexpr T bit_reverse(T x) noexcept
{
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
        result <<= 1;
        result |= x & 1;
        x >>= 1;
    }
    return result;
}
```

Reverses the bits of `x` so that the least significant bit becomes the most significant.

> EXAMPLE 1 ¶
> `bit_reverse(uint32_t{0x00001234})` equals `0x24c80000`.

§ 2.2. `bit_repeat`

```
template<unsigned_integral T>
constexpr T bit_repeat(T x, int length) noexcept(false)
{
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
        result |= ((x >> (i % length)) & 1) << i;
    }
    return result;
}
```

Repeats a pattern stored in the least significant `length` bits of `x`, as many times as fits into `x`.

> EXAMPLE 2 ¶
> `bit_repeat(uint32_t{0xc}, 4)` equals `0xcccccccc`.

§ 2.3. `next_bit_permutation` and `prev_bit_permutation`

```
template<unsigned_integral T>
constexpr T next_bit_permutation(T x) noexcept
{
    const int count = popcount(x);
    while (x != 0 && popcount(++x) != count) {}
    return x;
}
```

Returns the lowest number `y` in `(x, numeric_limits<T>::max()]` for which `popcount(y) == popcount(x)`, or `0` if none exists.

```
template<unsigned_integral T>
constexpr T prev_bit_permutation(T x) noexcept
{
```

```
    const int count = popcount(x);
    while (x != 0 && popcount(--x) != count) {}
    return x;
}
```

Returns the greatest number `y` in `[0, x)` for which `popcount(y) == popcount(x)`, or `0` if none exists.

EXAMPLE 3

The following loop can be used to generate $\binom{32}{3}$ elements of equal `popcount`:

```
for (uint32_t x = 0b111; x != 0; x = next_bit_permutation(x)) // ...
```

`x` will take the values:

```
dec:    7,   11,   13,   14,    19,    21, ...
bin:  111, 1011, 1101, 1110, 10011, 10101, ...
```

`prev_bit_permutation` can be used to generate this sequence in reverse.

2.4. `bit_compress`

```
template<unsigned_integral T>
constexpr T bit_compressr(T x, T m) noexcept
{
    T result = 0;
    for (int i = 0, j = 0; i < numeric_limits<T>::digits; ++i) {
        bool mask_bit = (m >> i) & 1;
        result |= (mask_bit & (x >> i)) << j;
        j += mask_bit;
    }
    return result;
}
```

For each one-bit in `m`, the corresponding bit in `x` is taken and packed contiguously into the result, starting with the least significant result bit.

```
template<unsigned_integral T>
constexpr T bit_compressl(T x, T m) noexcept
{
    return bit_reverse(bit_compressr(bit_reverse(x)), bit_reverse(m)));
}
```

For each one-bit in `m`, the corresponding bit in `x` is taken and packed contiguously into the result, starting with the most significant result bit.

EXAMPLE 4 ¶

```
x:
+---+---+---+---+
| a | b | c | d |
+---+---+---+---+
m:    |       |       bit_compressl(x, m):  bit_compressr(x, m):
+---+---+---+---+     +---+---+---+---+      +---+---+---+---+
| 0 | 1 | 0 | 1 |     | b | d | 0 | 0 |      | 0 | 0 | b | d |
+---+---+---+---+     +---+---+---+---+      +---+---+---+---+
```

NOTE: Intuitively, `m` is a "mask" that determines which bits of `x` are packed.

§ 2.5. `bit_expand`

```
template<unsigned_integral T>
constexpr T bit_expandr(T x, T m) noexcept
{
    T result = 0;
    for (int i = 0, j = 0; i < numeric_limits<T>::digits; ++i) {
        bool mask_bit = (m >> i) & 1;
        result |= (mask_bit & (x >> j)) << i;
        j += mask_bit;
    }
    return result;
}
```

For each one-bit in `m`, a bit from `x`, starting with the least significant bit is taken and shifted into the corresponding position of the `m` bit.

```
template<unsigned_integral T>
constexpr T bit_expandl(T x, T m) noexcept
{
    return bit_reverse(bit_expandr(bit_reverse(x)), bit_reverse(m)));
}
```

For each one-bit in `m`, a bit from `x`, starting with the most significant bit is taken and shifted into the corresponding position of the `m` bit.

EXAMPLE 5 ¶

```
x:
+---+---+---+---+
| a | b | c | d |
+---+---+---+---+
m:                    bit_expandl(x, m):    bit_expandr(x, m):
+---+---+---+---+     +---+---+---+---+      +---+---+---+---+
| 0 | 1 | 0 | 1 |     | 0 | a | 0 | b |      | 0 | c | 0 | d |
+---+---+---+---+     +---+---+---+---+      +---+---+---+---+
```

NOTE: Intuitively, `m` is a "mask" that determines where the bits of `x` are unpacked into.

§ 3. Motivation and scope

Bit-reversal, repetition, compression, and expansion are fundamental operations that meet multiple criteria which make them suitable for standardization:

1. They are common and useful operations.
2. They can be used to implement numerous other operations.
3. At least on some architectures, they have direct hardware support.
4. They are non-trivial to implement efficiently in software.
5. For known masks, numerous optimization opportunities are available.

`next_bit_permutation`, is the most niche of these operations and does not meet all criteria. However, I believe it is useful enough to be included in this proposal, and it fits the overall theme well.

§ 3.1. Applications

§ 3.1.1. Applications of `bit_reverse`

Bit-reversal is a common operation with uses in:

- **Cryptography**: scrambling bits
- **Networking**: as part of cyclic redundancy check computation
- **Graphics**: mirroring of images with one bit per pixel
- **Random number generation**: reversal can counteract low entropy of low-order bits such as in the case of linear congruential generators
- **Digital signal processing**: for radix-2 Cooley-Tukey FFT algorithms
- **Code obfuscation**: security by obscurity

§ 3.1.2. Applications of `bit_repeat`

The generation of recurring bit patterns is such a fundamental operation that it's hard to tie to any specific domain, but here are some use cases:

- **Debugging**: using obvious recurring bit pattenrs like `0xcccc...` to identify "garbage memory"
- **Bit manipulation**: generating alternating sequences of `1` and `0` for various algorithms
- **Eliminating integer divison**: for `x >> (i % N)` with very small `N`, `bit_repeat(x, N) >> i` can be used instead
- **Testing**: recurring bit patterns can make for good test cases when implementing numerics
- **Error detection**: known bit patterns can be introduced to spot failed transmission

§ 3.1.3. Applications of `bit_compress` and `bit_expand`

Compression and expansion are also common, with uses in:

- **Space-filling curves**: Morton/Z-Order and Hilbert curves
- **Input/output**: especially for variable-length encodings, such as UTF-8 (§ 3.2.3 UTF-8 decoding with bit_compressr)
- **Chess engines**: for bitboards; see [ChessProgramming1]
- **Genomics**: according to [ARM1]

A GitHub code search for `/(_pdep_u|_pext_u)(32|64)/ AND language:c++` reveals ~1300 files which use the intrinsic wrappers for the x86 instructions.

3.1.4. Applications of `next_bit_permutation`

`next_bit_permutation` is useful for iterating over all subsets with a fixed amount of elements (see below for an example).

Unlike algorithms such as `next_permutation`, it can be used with no modification of the original range. Together with `prev_bit_permutation`, it is possible to create a bidirectional view of all fixed-size subsets of a range.

3.2. Motivating examples

3.2.1. Implementing `countr_zero` with `bit_repeat`

[Anderson1] contains a vast amount of algorithms, many of which involve masks of alternating `0`s and `1`s.

When written as "magic numbers" in code, these masks can make it quite hard to understand the overall pattern and to generalize these algorithms. `bit_repeat` allows one to be more expressive here:

EXAMPLE 6 ¶

```
unsigned int v;      // 32-bit word input to count zero bits on right
unsigned int c = 32; // c will be the number of zero bits on the right
v &= -v;
if (v) c--;
if (v & 0x0000FFFF bit_repeat((1 << 16) - 1, 32)) c -= 16;
if (v & 0x00FF00FF bit_repeat( (1 << 8) - 1, 16)) c -=  8;
if (v & 0x0F0F0F0F bit_repeat( (1 << 4) - 1,  8)) c -=  4;
if (v & 0x33333333 bit_repeat( (1 << 2) - 1,  4)) c -=  2;
if (v & 0x55555555 bit_repeat( (1 << 1) - 1,  2)) c -=  1;
```

It is now obvious how this can be expressed in a loop:

```
// ...
for (int i = 16; i != 0; i /= 2) {
    unsigned mask = bit_repeat((1u << i) - 1, i * 2);
    if (v & mask) c -= i;
}
```

`bit_repeat` has been an invaluable asset in the implementation of the remaining functions in this proposal (see [Schultke1] for details).

3.2.2. Interleaving bits with `bit_expand`

A common use case for expansion is interleaving bits. This translates Cartesian coordinates to the index on a Z-order curve. Space filling curves are a popular technique in compression.

EXAMPLE 7

```
unsigned x = 3; // 0b011
unsigned y = 5; // 0b101
const auto i = bit_expandr(x, bit_repeat(0b10u, 2)) // i = 0b01'10'11
             | bit_expandr(y, bit_repeat(0b01u, 2));
```

3.2.3. UTF-8 decoding with `bit_compressr`

`bit_compress` and `bit_expand` are useful in various I/O-related applications. They are particularly helpful for dealing with variable-length encodings, where "data bits" are interrupted by bits which signal continuation of the data.

EXAMPLE 8

The following code reads 4 bytes from some source of UTF-8 data and returns the codepoint. For the sake of simplicity, let's ignore details like reaching the end of the file, or I/O errors.

```
uint_least32_t x = load32_little_endian(utf8_data_pointer);
switch (countl_one(uint8_t(x))) {
case 0: return bit_compressr(x, 0b01111111);
case 1: /* error */;
case 2: return bit_compressr(x, 0b00111111'00011111);
case 3: return bit_compressr(x, 0b00111111'00111111'00001111);
case 4: return bit_compressr(x, 0b00111111'00111111'00111111'00000111);
}
```

3.2.4. Other operations based on `bit_compress` and `bit_expand`

Many operations can be built on top of `bit_compress` and `bit_expand`. However, direct hardware support is often needed for the proposed functions to efficiently implement them. Even without such

support, they can be canonalized into a faster form. The point is that `bit_compress` and `bit_expand` allow you to *express* these operations.

EXAMPLE 9 ¶

```
// x & 0xf
bit_expandr(x, 0xf)
bit_compressr(x, 0xf)

// (x & 0xf) << 4
bit_expandr(x, 0xf0)
// (x >> 4) & 0xf
bit_compressr(x, 0xf0)

// Clear the least significant one-bit of x.
x ^= bit_expandr(1, x)
// Clear the nth least significant one-bit of x.
x ^= bit_expandr(1 << n, x)
// Clear the n least significant one-bits of x.
x ^= bit_expandr((1 << n) - 1, x)

// (x >> n) & 1
bit_compressr(x, 1 << n)
// Get the least significant bit of x.
bit_compressr(x, x) & 1
// Get the nth least significant bit of x.
(bit_compressr(x, x) >> n) & 1

// popcount(x)
countr_one(bit_compressr(-1u, x))
countr_one(bit_compressr(x, x))
```

§ 3.2.5. Iterating over subsets using `next_bit_permutation`

EXAMPLE 10 ¶

```
const char set[] {'a', 'b', 'c', 'd'};

for (unsigned bits = 0b11; bits < 16; bits = next_bit_permutation(bits)) {
    if (bits & 1) std::print("{}", set[0]);
    if (bits & 2) std::print("{}", set[1]);
    if (bits & 4) std::print("{}", set[2]);
    if (bits & 8) std::print("{}", set[3]);
    std::print(" ");
}
```

This code prints:

```
ab ac bc ad bd cd
```

§ 3.3. Hardware support

Operation	x86_64	ARM	RISC-V
bit_reverse	(BSWAP)	RBIT^{SVE2}	(vrgather^{V})
bit_repeat			
next_bit_permutation	(TZCNT^{BMI}/BSF)	(CTZ)	(ctz^{B})
prev_bit_permutation	(TZCNT^{BMI}/BSF)	(CTZ)	(ctz^{B})
bit_compressr	PEXT^{BMI2}	BEXT^{SVE2}	(vcompress^{V})
bit_expandr	PDEP^{BMI2}	BDEP^{SVE2}	(viota+vrgather^{V})
bit_compressl	(PEXT^{BMI2}+POPCNT^{ABM})	BGRP^{SVE2}, (BEXT^{SVE2}+CNT^{SVE})	(vcompress^{V})
bit_expandl	(PDEP^{BMI2}+POPCNT^{ABM})	(BDEP^{SVE2}+CNT^{SVE})	(viota+vrgather^{V})

(Parenthesized) entries signal that the instruction does not directly implement the function, but greatly assists in its implementation.

NOTE: The RISC-V instructions marked V are all vector instructions. It is possible to turn each bit into a vector element, perform the wanted operation, and compress back into a bit-vector.

3.3.1. Support for `bit_reverse`

This operation is directly implemented in ARM through `RBIT`.

Any architecture with support for `byteswap` (such as x86 with `BSWAP`) also supports bit-reversal in part. [Warren1] presents an O(log n) algorithm which operates by swapping lower and upper `N / 2`, ..., `16`, `8`, `4`, `2`, and `1` bits in parallel. Byte-swapping implements these individual swaps up to 8 bits, requiring only three more parallel swaps in software:

```
// assuming a byte is an octet of bits, and assuming the width of x is a power
x = byteswap(x);
x = (x & 0x0F0F0F0F) <<  4 | (x & 0xF0F0F0F0) >>  4; // ... quartets of bits
x = (x & 0x33333333) <<  2 | (x & 0xCCCCCCCC) >>  2; // ... pairs of bits
x = (x & 0x55555555) <<  1 | (x & 0xAAAAAAAA) >>  1; // ... individual bits
```

It is worth noting that clang provides a cross-platform family of intrinsics. `__builtin_bitreverse` uses byte-swapping or bit-reversal instructions if possible.

Such an intrinsic has been requested from GCC users a number of times in [GNU1].

3.3.2. Support for `bit_repeat`

To my knowledge, `bit_repeat` has no direct support in the general case. However, for specific cases like `bit_repeat(x, 8)`, SIMD permutation/duplication/gather/broadcast instructions can be used.

The primary use of `bit_repeat` is to express repeating bit patterns without magic numbers, i.e. to improve code quality. It is worth noting that the reference implementation ([Schultke1]) requires only O(log N) fundamental bitwise operations.

3.3.3. Support for `next_bit_permutation`

The next permutation can be efficiently obtained if the platform has a "count trailing zeros" instruction. Integer division can also be used as a fallback, although software computation of trailing zeros may be faster than integer division.

Both options are shown in [Anderson1], and one is implemented in [Schultke1].

`prev_bit_permutation` can be accelerated using the same operation.

§ 3.3.4. Support for `bit_compress` and `bit_expand`

Starting with Haswell (2013), Intel CPUs directly implement compression and expansion with with `PEXT` and `PDEP` respectively. AMD CPUs starting with Zen 3 implement `PDEP` and `PEXT` with 3 cycles latency, like Intel. Zen 2 and older implement `PDEP`/`PEXT` in microcode, with 18 cycles latency.

ARM also supports these operations directly with `BDEP`, `BEXT`, and `BGRP` in the SVE2 instruction set. [Warren1] mentions other older architectures with direct support.

Overall, only recent instruction set extensions offer this functionality directly. However, when the mask is a constant, many different strategies for hardware acceleration open up. For example

- interleaving bits can be assisted (though not fully implemented) using ARM `ZIP1`/`ZIP2`
- other permutations can be assisted by ARM `TBL` and `TBX`

As [Warren1] explains, the cost of computing `bit_compress` and `bit_expand` in software is dramatically lower for a constant mask. For specific known masks (such as a mask with a single one-bit), the cost is extremely low.

All in all, there are multiple factors that strongly suggest a standard library implementation:

1. The strategy for computing `bit_compress` and `bit_expand` depends greatly on the architecture and on information about the mask, even if the exact mask isn't known.
    - `TZCNT`, `CLMUL` (see [Schultke1] or [Zp7] for specifics), and `POPCNT` are helpful.
2. ISO C++ does not offer a mechanism through which all of this information can be utilized. Namely, it is not possible to change strategy based on information that only becomes available during optimization passes. Compiler extensions such as `__builtin_constant_p` offer a workaround.
3. ISO C++ does not offer a mechanism through which function implementations can be chosen based on the surrounding context. In a situation where multiple `bit_compress` calls with the same mask `m` are performed, it is significantly faster to pre-compute information based on the mask once, and utilize it in subsequent calls. The same technique can be used to accelerate integer division for multiple divisons with the same divisor.

Bullets 2. and 3. suggest that `bit_compress` and `bit_expand` benefit from being implemented directly in the compiler via intrinsic, even if hardware does not directly implement these operations.

Even with a complete lack of hardware support, a software implementation of `compress_bitsr` in [Schultke1] emits essentially optimal code if the mask is known.

EXAMPLE 11 ¶

```
unsigned bit_compress_known_mask(unsigned x) {
    return cxx26bp::bit_compressr(x, 0xf0f0u);
}
```

Clang 18 emits the following (and GCC virtually the same); see [CompilerExplorer1]:

```
bit_compress_known_mask(unsigned int): # bit_compress_known_mask(unsigned in
        mov     eax, edi               # {   unsigned int eax = edi;
        shr     eax, 4                 #     eax >>= 4;
        and     eax, 15                #     eax &= 0xf;
        shr     edi, 8                 #     edi >>= 8;
        and     edi, 240               #     edi &= 0xf0;
        or      eax, edi               #     eax |= edi;
        ret                            #     return eax; }
```

Knowing the implementation of `bit_compressr`, this feels like dark magic. This is an optimizing compiler at its finest hour.

§ 4. Impact on existing code

This proposal is purely a standard library expansion. No existing code is affected.

§ 5. Design considerations

The design choices in this paper are based on [P0553R4], wherever applicable.

§ 5.1. Signature of `bit_repeat`

`bit_repeat` follows the "use `int` if possible" rule mentioned in [P0553R4]. Other functions such as `std::rotl` and `std::rotr` also accept an `int`.

It is also the only function not marked `noexcept`. It does not throw, but it is not `noexcept` due to its narrow contract (Lakos rule).

§ 5.2. Why the names *compress* and *expand*?

The use of `compress` and `expand` is consistent with the mask-based permutations for `std::simd` proposed in [P2664R6].

Furthermore, there are multiple synonymous sets of terminology:

1. `deposit` and `extract`
2. `compress` and `expand`
3. `gather` and `scatter`

I have decided against `deposit` and `extract` because of its ambiguity:

Taking the input `0b10101` and densely packing it to `0b111` could be described as:

> Extract each second bit from `0b10101` and densely deposit it into the result.

Similarly, taking the input `0b111` and expanding it into `0b10101` could be described as:

> Extract each bit from `0b111` and sparsely deposit it in the result.

Both operations can be described with `extract` and `deposit` terminology, making it virtually useless for keeping the operations apart. `gather` and `scatter` are simply the least common way to describe these operations, which makes `compress` and `expand` the best candidates.

Further design choices are consistent with [P0553R4]. The abbreviations `l` and `r` for left/right are consistent with `rotl`/`rotr`. The prefix `bit_` is consistent with `bit_floor` and `bit_ceil`.

§ 5.3. Why the lack of generalization?

§ 5.3.1. No generalized `bit_compress` and `bit_expand`

[N3864] originally suggested much more general versions of compression and expansion, which support:

1. performing the operation not just on the whole operand, but on "words" of it, in parallel
2. performing the operation not just on bits, but on arbitrarily sized groups of bits

I don’t propose this generality for the following reasons:

1. The utility functions in `<bit>` are not meant to provide a full bitwise manipulation library, but fundamental operations, especially those that can be accelerated in hardware while still having reasonable software fallbacks.

2. These more general form can be built on top of the proposed hardware-oriented versions. This can be done with relative ease and with little to no overhead.
3. The generality falsely suggests hardware support for all forms, despite the function only being accelerated for specific inputs. This makes the performance characteristics unpredictable.
4. The proposed functions have wide contracts and can be `noexcept` (Lakos rule). Adding additional parameters would likely require a narrow contract.
5. Generality adds complexity to the standardization process, to implementation, and from the perspective of language users. It is unclear whether this added complexity is worth it in this case.

5.3.2. No generalized `bit_reverse`

Bit reversal can also be generalized to work with any group size:

```
template <typename T>
T bit_reverse(T x, int group_size = 1) noexcept(false);
```

With this generalization, `byteswap(x)` on conventional platforms is equivalent to `bit_reverse(x, 8)`.

However, this general version is much less used, not as consistently supported in hardware, and has a narrow contract. `group_size` must be a nonzero factor of `x` for this operation to be meaningful.

Therefore, a generalized bit-reversal is not proposed in this paper.

5.4. Why does the signature of `bit_compress` require two same `T`s?

Initially, I went through a number of different signatures.

```
template<unsigned_integral T, unsigned_integral X>
constexpr T bit_compressr(X x, T m) noexcept;
```

This signature is quite clever because the result never has more bits than the mask `m`. However, it is surprising that the mask plays such a significant role here.

Furthermore, I've realized that while the result never has more bits than `m`, `bit_compressl` must still deposit bits starting with the most significant bits of the result. This suggests the following:

```
template<unsigned_integral T, unsigned_integral X>
constexpr common_type_t<T, X> bit_compressr(X x, T m) noexcept;
```

However, it is not trivial to juggle bits between the left and right versions of `bit_compress` and `bit_expand`. The behavior is also not intuitive when a zero-extension occurs. For wider `x`, the mask is always zero-extended to the left, which makes the left and right versions slightly asymmetrical.

Since this proposal includes low-level bit operations, it is reasonable and safe to require the user to be explicit. A call to `bit_compress` or `bit_expand` with two different types is likely a design flaw or bug. Therefore, I have settled on the very simple signature:

```
template<unsigned_integral T>
constexpr T bit_compressr(T x, T m) noexcept;
```

6. Possible implementation

6.1. Reference implementation

All proposed functions have been implemented in [Schultke1]. This reference implementation is compatible with all three major compilers, and leverages hardware support from ARM and x86_64 where possible.

6.2. Other implementations

[Warren1] presents algorithms which are the basis for [Schultke1].

- An O(log n) `bit_reverse`
- An O($\log^2$ n) `bit_compress` and `bit_expand`
  - can be O(log n) with hardware support for carry-less multiplication aka. GF(2) polynomial multiplication

[Zp7] offers fast software implementations for PEXT and PDEP, optimized for x86_64.

[StackOverflow1] contains discussion of various possible software implementations of `bit_compressr` and `bit_expandr`.

7. Proposed wording

The proposed changes are relative to the working draft of the standard as of [N4917].

§ 7.1. Feature-testing

In subclause 17.3.2 [version.syn] paragraph 2, update the synopsis as follows:

```
#define __cpp_lib_bitops                    ~~201907L~~20XXXXL // freestanding, a
```

§ 7.2. Header synopsis

In subclause 22.15.2 [bit.syn], update the synopsis as follows:

```
+ // 22.15.X, permutations
+ template<class T>
+   constexpr T bit_reverse(T x) noexcept;
+ template<class T>
+   constexpr T bit_repeat(T x, int l);
+ template<class T>
+   constexpr T next_bit_permutation(T x) noexcept;
+ template<class T>
+   constexpr T prev_bit_permutation(T x) noexcept;
+ template<class T>
+   constexpr T bit_compressr(T x, T m) noexcept;
+ template<class T>
+   constexpr T bit_expandr(T x, T m) noexcept;
+ template<class T>
+   constexpr T bit_compressl(T x, T m) noexcept;
+ template<class T>
+   constexpr T bit_expandl(T x, T m) noexcept;
```

§ 7.3. New subclause

In subclause 22.15 [bit], add the following subclause:

22.15.X Permutation [bit.permute]

1 In the following descriptions, let *N* denote `numeric_limits<T>::digits`. Let α_n denote the *n*-th least significant bit of `α`, so that `α` equals $\sum_{n=0}^{N-1} \alpha_n 2^n$

```
template<class T>
  constexpr T bit_reverse(T x) noexcept;
```

2 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

3 *Returns*: $\sum_{n=0}^{N-1} x_n 2^{N-n-1}$

[*Note*: `bit_reverse(bit_reverse(x))` equals `x`. — *end note*]

```
template<class T>
  constexpr T bit_repeat(T x, int l);
```

4 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

5 *Preconditions*: `l` is greater than zero.

6 *Returns*: $\sum_{n=0}^{N-1} x_{(n \mod l)} 2^N$

7 *Throws*: Nothing.

```
template<class T>
  constexpr T next_bit_permutation(T x) noexcept;
```

8 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

9 *Returns*: The lowest integer `y` in (`x`, `numeric_limits<T>::max()]` for which `popcount(x)` equals `popcount(y)`, or `0` if none exists.
[*Note*: If `x` is a power of two or zero, `next_bit_permutation(x)` equals `x << 1`. — *end note*]

```
template<class T>
  constexpr T prev_bit_permutation(T x) noexcept;
```

10 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

11 *Returns*: The greatest integer `y` in [`0`, `x`) for which `popcount(x)` equals `popcount(y)`, or `0` if none exists.

[*Note*: If `x == 0 || next_bit_permutation(x) != 0` is `true`, `prev_bit_permutation(next_bit_permutation(x))` equals `x`. *end note*]

```
template<class T>
  constexpr T bit_compressr(T x, T m) noexcept;
```

12 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

13 *Returns*: $\sum_{n=0}^{N-1} m_n x_n 2^{(\sum_{k=0}^{n-1} m_k)}$

```
template<class T>
  constexpr T bit_expandr(T x, T m) noexcept;
```

14 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

15 *Returns*: $\sum_{n=0}^{N-1} m_n x_{(\sum_{k=0}^{n-1} m_k)} 2^n$

[*Note*: If `x & ~m` equals zero, then `bit_expandr(bit_compressr(x, m), m)` equals `x`. If `x >> popcount(m)` equals zero, then `bit_compressr(bit_expandr(x, m), m)` equals `x`. — *end note*]

```
template<class T>
  constexpr T bit_compressl(T x, T m) noexcept;
```

16 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

17 *Returns*: `bit_reverse(bit_compressr(bit_reverse(x), bit_reverse(m)))`.

```
template<class T>
  constexpr T bit_expandl(T x, T m) noexcept;
```

18 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

19 *Returns*: `bit_reverse(bit_expandr(bit_reverse(x), bit_reverse(m)))`.

NOTE: I would have preferred a less mathematical approach to defining these functions. However, it is too difficult to precisely define `bit_compress` and `bit_expand` without visual aids, pseudo-code, or other crutches.

§ 8. Acknowledgements

I greatly appreciate the assistance of Stack Overflow users in assisting me with research for this proposal. I especially thank Peter Cordes for his tireless and selfess dedication to sharing knowledge.

I also thank various Discord users from Together C & C++ and #include<C++> who have reviewed drafts of this proposal and shared their thoughts.

§ References

§ Normative References

[N4917]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022. URL: https://wg21.link/n4917

§ Informative References

[Anderson1]
Sean Eron Anderson. *Bit Twiddling Hacks*. URL: https://graphics.stanford.edu/~seander/bithacks.html

[ARM1]
Arm Developer Community. *Introduction to SVE2, Issue 02, Revision 02*. URL: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/102340_0001_02_en_introduction-to-sve2.pdf?revision=b208e56b-6569-4ae2-b6f3-cd7d5d1ecac3

[ChessProgramming1]
VA. *chessprogramming.org/BMI2, Applications*. URL: https://www.chessprogramming.org/BMI2#Applications

[CompilerExplorer1]
Jan Schultke. *Compiler Explorer example for bit_compressr*. URL: https://godbolt.org/z/5dcTjE5x3

[GNU1]
Marc Glisse et al.. *Bug 50481 - builtin to reverse the bit order*. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=50481

[N3864]
Matthew Fioravante. *A constexpr bitwise operations library for C++*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3864.html

[P0553R4]
Jens Maurer. *Bit operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0553r4.html

[P2664R6]

Daniel Towner; Ruslan Arutyunyan. *Extend std::simd with permutation API*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2664r6.html#permute_by_mask

[Schultke1]

Jan Schultke. *C++26 Bit Permutations*. URL: https://github.com/Eisenwave/cxx26-bit-permutations

[StackOverflow1]

Jan Schultke et al.. *What is a fast fallback algorithm which emulates PDEP and PEXT in software?*. URL: https://stackoverflow.com/q/77834169/5740428

[Warren1]

Henry S. Warren, Jr.. *Hacker's Delight, 2nd Edition*.

[Zp7]

Zach Wegner. *Zach's Peppy Parallel-Prefix-Popcountin' PEXT/PDEP Polyfill*. URL: https://github.com/zwegner/zp7